

09.07.2024

Übungsblatt 11

Speichern Sie Ihre Lösungen bis spätestens 16.07.2024 18:00 Uhr auf Ihrem Gruppen-Übungsserver `scilab-00???.cs.uni-kl.de`.

Die Abgabe erfolgt indem Sie zusätzlich die **Lösungsdateien (als ZIP-File) auf OLAT hochladen**.

Bitte geben Sie bei jeder Abgabe mit an **welche Gruppenmitglieder*innen aktiv an der Bearbeitung des aktuellen Übungsblattes mitgewirkt haben**. Fügen Sie dazu dem auf OLAT hochzuladenden ZIP-File eine zusätzliche Datei "TeamBlatt11.txt" mit den Namen der aktiven Gruppenmitglieder*innen hinzu.

Falls Sie Fragen haben nutzen Sie bitte das Forum in OLAT oder schicken Sie eine Mail direkt an w2t@cs.uni-kl.de.

Viel Erfolg!

1. Aufgabe: Django – Queries (9 P)

Schreiben Sie nun in der Django-Shell die Anweisungen um folgende Queries durch geeignete QuerySet-Ausdrücke zu realisieren:

- a) alle Studenten deren Namen mit 'F' beginnt
- b) alle Studenten deren Namen mit 'F' beginnt und deren Matrikelnummer größer 27000 ist
- c) alle Studenten deren Namen mit 'F' beginnt und nicht mit 'e' endet.
- d) die Professoren die nicht einen Studenten mit dem Namen 'Fichte' unterrichten
- e) alle Studierenden die eine der Matrikelnummern 26120, 27103, 29999 haben, absteigend sortiert nach „Name“ (benutzen Sie *in*)
- f) alle Studierenden die eine der Matrikelnummern 26120, 27103, 29999 haben, aufsteigend sortiert nach „Name“ (benutzen Sie *Q-Objekte*)
- g) Achtung erweitern Sie zuerst das Modell, so dass folgende Query möglich ist:
alle Vorlesungen die von „Prof. Wirth“ gelesen werden
- h) alle Vorlesungen die von „Dr. Wirth“ gelesen werden
- i) Für Fortgeschrittene: alle Studierende die keine Vorlesung besuchen

Hinweis: Speichern Sie die Anfrage und das Ergebnis in die Abgabedatei `~/django/B11A1.txt`

2. Aufgabe: Django – Codeanalyse

(1+2+2= 5 P)

Im Folgenden gehen wir davon aus, dass sie ihr Projekt „University“ und ihre App „pruefungsamt“ analog zu den Beispielen aus den Folien benannt haben. Falls dies nicht der Fall ist nutzen Sie stattdessen Ihre Bezeichnungen.

Finden und korrigieren Sie die Fehler in folgenden Django-Code-Fragmenten.

(Kommentieren Sie jede Zeile! Erläutern Sie kurz wodurch der Fehler verursacht wurde!)

a)

```
from pruefungsamt.models import *  
s = Student(matnr=4711)  
s.hoert = Vorlesung.object.all()
```

b)

```
from pruefungsamt.models import *  
Student.objects.filter(matnr=4711).delete()  
s = Student(name="Timmy", matnr='4711')  
s.save()  
renamed=lambda s,i,o: s.name.replace(i,o)  
s.name=renamed(s,'i','o')  
s.save()
```

c)

```
from pruefungsamt.models import *  
assert Student.objects.filter(matnr=4711).exists()  
s = Student.objects.get(matnr=4711)  
print(Student.objects.count(), s.pk)  
s.pk = None  
renamed=lambda s,i,o: s.name.replace(i,o)  
s.name=renamed(s,'o','a')  
s.save()  
print(Student.objects.count(), s.pk)
```

3. Aufgabe: Django – Views

(8 P)

Implementieren Sie [Views](#)¹ für unser Django-Projekt „University“. um folgende Features zu realisieren. In der Datenbank sollten mindestens die Daten aus dem [Wikipedia-SQL-Beispiel](#)² vorhanden sein.

Die Template-Engine darf in dieser Aufgabe noch **nicht** verwendet werden.

- a) Unter dem URL-Pfad **/service/anzahl/aktiveStudenten** soll eine Seite aufgerufen werden können, die die Anzahl der aktiven Studenten anzeigt, also die, die mindestens eine Vorlesung hören. Implementieren Sie dazu in **views.py** eine Methode **show_aktive_studenten_anzahl(request)**, welche als Response im *Text*-Format die Anzahl der Studierende ausgibt, die mindestens eine Vorlesung hören.
(Denken Sie daran immer den passenden [mime-Type](#)³ im Response zu verwenden.)
- b) Unter dem URL-Pfad **/professoren** soll eine Seite aufgerufen werden können, die alle Professoren anzeigt. Implementieren Sie dazu in **views.py** eine Methode **show_professoren_all(request)**, welche als Response im *HTML*-Format (validierbar korrekt!) in einer Tabelle zeilenweise alle Professoren (Attribute als Tabellenspalten) aufsteigend sortiert nach Namen ausgibt.
- c) Unter dem URL-Pfad **/professoren/json** soll die Liste der Professoren im JSON-Format ausgegeben werden. (Sie könne das JSON-Format selber erzeugen oder eine entsprechende Python- oder Django-Funktion benutzen.) Implementieren Sie dazu in **views.py** eine Methode **show_professoren_all_as_json(request)**.
(Denken Sie daran immer den passenden [mime-Type](#) im Response zu verwenden.)

1 Django-Views: <https://docs.djangoproject.com/en/4.2/topics/http/views/>
2 Daten für Datenbank: <https://de.wikipedia.org/wiki/SQL>
3 MIME Type-Referenz: <http://wiki.selfhtml.org/wiki/Referenz:MIME-Typen>

4. Aufgabe: Django – Templates

(8 P)

Wir wollen nun die Template-Engine zur Lösung benutzen.

- a) Realisieren Sie Aufgabe 3 nun nochmals, diesmal aber mit Hilfe der von Django zur Verfügung gestellten Template-Engine⁴. Die vorherige Implementierung bleibt aber erhalten. Um beide parallel benutzen zu können, fügen Sie in die URL-Pfade zu dieser Aufgabe den Präfix '/x/' ein und in die View-Namen den Präfix 'x_'.
Nutzen Sie den for...empty⁵-Template-Tag um übergebene Listen oder Query-Sets im Template abzuarbeiten. Wenn eine Liste leer ist, soll ein entsprechender Text ausgegeben werden.
- b) Unter dem URL-Pfad / soll zudem eine Index-Seite mit Links auf die Unterseiten erscheinen, die sie anbieten möchten (Inhalt aus einem Template). Nutzen Sie zur Generierung der URLs in den Links das Tag {% URL 'name' %}⁶.

4 Template-Engine: <https://docs.djangoproject.com/en/4.2/topics/templates/>
5 „for...empty“-Template-Tag: <https://docs.djangoproject.com/en/4.2/ref/templates/builtins/#for-empty>
6 „url“-Template-Tag: <https://docs.djangoproject.com/en/4.2/ref/templates/builtins/#url>